EdGame Analytics Platform — Technical Progress Summary

Author: Yousef Radwan **Course:** TIE 251 CCS **Date:** March 2026 **Repository:** github.com/yradwan147/EdGame

---

Executive Summary

EdGame is an educational game-based analytics platform that captures telemetry from student gameplay and surfaces actionable learning insights for teachers. The project has completed **Sprint 1 (Architecture & Design)** and **Sprint 2 (Alpha Implementation)**, establishing a fully documented architecture and a working codebase deployed as a monorepo.

Sprint 1 Deliverables — Architecture & Technical Planning

Sprint 1 focused on producing the foundational architecture decisions, data model, API specification, and assessment framework that guide all implementation work.

1. Architecture Decision Record (ADR)

Formal record of 6 key technology and architecture decisions with rationale, alternatives considered, and consequences.

Decision	Choice	Why
Web Framework	Next.js 14 (App Router)	Server components, API routes, SSR in one framework
Game Engine	KAPLAY.js	Lightweight 2D engine already proven in Pulse Realms prototype
Database	SurrealDB	Multi-model (document + graph + relational), easy self-hosting
Hosting	Railway	Simple container deployment, integrated databases
Monorepo	pnpm + Turborepo	Shared packages, parallel builds, single repo
Scaling	Phased approach	Start with 1 game, expand to 5 over course lifecycle

File: `docs/adr/001-phase1-architecture.pdf`

2. Data Model

Complete database schema with 12 tables covering the full data lifecycle — from schools and users through game sessions and telemetry events to computed analytics metrics.

Tables: `schools`, `users`, `classes`, `class_members`, `game_environments`, `assignments`, `game_sessions`, `game_events`, `computed_metrics`, `student_metrics_daily`, `student_insights`, `questions`

Each table includes field definitions, types, defaults, indexes, and row-level permissions (RBAC).

File: `docs/architecture/data-model.surql`

3. REST API Specification

Complete API route inventory with request/response JSON shapes for all endpoints:

Category	Routes	Operations
Authentication	2	Login, Logout
Game Environments	1	List active environments
Assignments	4	CRUD (Create, Read, Update, Delete)
Game Sessions	3	Create, Get, Update
Telemetry Events	1	Batch insert (up to 200 events/request)
Analytics	2	Class-level and student-level metrics

File: `docs/architecture/api-routes.pdf`

4. Monorepo Structure

Directory layout specification for the full platform, defining workspace packages and their responsibilities.

```
apps/web/           → Next.js dashboard + API
apps/games/         → Game builds (Pulse Realms + 4 future games)
packages/shared/    → Database client, types, validators
packages/ui/        → Reusable UI component library
db/                 → Migration and seed scripts
```

File: `docs/architecture/monorepo-structure.pdf`

5. Assessment Framework (Evidence-Centered Design)

The analytics engine is grounded in Evidence-Centered Design (ECD), a psychometric framework that connects observable game behaviors to measurable learning constructs.

5a. Six Analytics Dimensions

Every metric maps to one of six pedagogically grounded dimensions:

#	Dimension	What it measures
D1	Cognitive Proficiency	Knowledge accuracy, mastery progression
D2	Engagement & Flow	Session duration, voluntary replay, focus
D3	Strategic Thinking	Planning, resource management, adaptation
D4	Social-Collaborative	Teamwork, communication, peer support
D5	Socio-Emotional Learning	Resilience, emotional regulation, persistence
D6	Temporal Dynamics	Learning rate, improvement trends over time

5b. Coverage Matrix

Maps which of the 5 planned games cover which dimensions, ensuring portfolio-wide coverage:

File: `docs/assessment/coverage-matrix.json`

5c. Metric-to-Telemetry Mapping

Maps 15 core analytics metrics to the exact game telemetry events that produce them, with code-level references to the source files.

File: `docs/assessment/metric-mapping.json`

5d. ECD Models per Game

Full Evidence-Centered Design breakdown for each game with four models:

- **Competency Model** — what constructs are being assessed
- **Evidence Model** — what telemetry events serve as evidence, with scoring rules
- **Task Model** — what game mechanics produce observable behaviors
- **Assembly Model** — how raw data is aggregated into per-student metrics

Game	Primary Dimension	Status
Pulse Realms	D1 Cognitive	Full ECD model
Concept Cascade	D3 Strategic	Draft skeleton
Lab Explorer	D3 Strategic	Draft skeleton
Knowledge Quest	D5 SEL	Draft skeleton
Survival Equation	D4 Social	Draft skeleton

Directory: `docs/assessment/ecd/` **Full model:** `docs/assessment/ecd/pulse-realms.pdf`

Sprint 2 Deliverables — Alpha Implementation

Sprint 2 translated the Sprint 1 architecture into working code across 5 issues.

6. Monorepo Scaffold (Issue #8)

Working pnpm + Turborepo monorepo with:

- Next.js 14 web app with Tailwind CSS
- Shared TypeScript package (DB client, types, validators)
- UI component library (Button, Card)
- Health check endpoint (`GET /api/health`)

Key files:

- `pnpm-workspace.yaml`
- `turbo.json`
- `apps/web/` — Next.js app
- `packages/shared/` — Shared types, DB client, Zod validators
- `packages/ui/` — UI components

7. Database Schema & Seed Data (Issue #9)

Migration script implementing the full 12-table schema in SurrealQL, plus seed data for development (1 school, 1 teacher, 5 students, 14 questions, 1 class, 1 assignment).

Files:

- `db/migrations/001_initial_schema.surql`
- `db/seeds/001_seed.surql`

8. Pulse Realms Game Migration (Issue #10)

Migrated the existing Pulse Realms prototype into the monorepo and wired it to the platform backend:

- Telemetry system now flushes events to the API every 10 seconds
- Question engine can load questions from the database (with offline JSON fallback)
- Game sessions are created/closed via API on start/end

Directory: `apps/games/pulse-realms/` **Key modified files:**

- `apps/games/pulse-realms/src/systems/telemetry.js`
- `apps/games/pulse-realms/src/systems/questionEngine.js`
- `apps/games/pulse-realms/main.js`

9. Teacher Dashboard UI (Issue #11)

5 functional dashboard pages with sidebar navigation:

Page	Purpose
Teacher Home	Class list with quick stats
Class View	Student roster, sortable metrics table
Student Detail	Top-3 insights, 6-dimension progress bars, session timeline
Assignment Creator	Form for creating game-based assignments
Analytics Overview	Concept mastery heatmap, engagement trends, at-risk alerts

Directory: `apps/web/app/(dashboard)/`

10. API Routes (Issue #12)

11 REST API route handlers implementing the Sprint 1 API specification:

- Sessions: `POST`, `GET /:id`, `PATCH /:id`
- Events: `POST /:id/events` (batch, up to 200)
- Assignments: `GET`, `POST`, `GET /:id`, `PUT /:id`, `DELETE /:id`
- Environments: `GET`
- Questions: `GET ?subject=`
- Analytics: `GET /class/:id`, `GET /student/:id`

All routes include Zod schema validation and SurrealDB integration.

Directory: `apps/web/app/api/` **Validators:** `packages/shared/src/validators/`

Repository Structure Overview

```
EdGame/
├── docs/                              # Architecture & design documentation
│   ├── adr/                           #   Architecture Decision Records
│   ├── architecture/                  #   Data model, API spec, monorepo layout
│   └── assessment/                    #   ECD models, coverage matrix, metrics
├── apps/
│   ├── web/                           # Next.js 14 dashboard + API
│   │   ├── app/(dashboard)/           #   5 teacher dashboard pages
│   │   └── app/api/                   #   11 REST API route handlers
│   └── games/
│       └── pulse-realms/              # KAPLAY.js educational combat game
├── packages/
│   ├── shared/                        # DB client, TypeScript types, Zod validators
│   └── ui/                            # Reusable UI components (Button, Card)
├── db/
│   ├── migrations/                    # SurrealQL schema (12 tables)
│   └── seeds/                         # Development seed data
├── pnpm-workspace.yaml
└── turbo.json
```

What's Next — Sprint 3 Targets

- Deploy to Railway (web app + SurrealDB instance)
- Implement authentication (NextAuth.js with role-based access)
- Connect dashboard pages to live API data
- Run Pulse Realms within the platform with real telemetry capture
- Begin analytics computation pipeline (session metrics, daily rollups)

Quick Reference: All Documentation Files

Document	Link	Purpose
ADR	001-phase1-architecture.pdf	6 architecture decisions with rationale
Data Model	data-model.surql	12-table SurrealQL schema
API Spec	api-routes.pdf	Full REST API with request/response shapes
Monorepo Layout	monorepo-structure.pdf	Directory structure specification
Coverage Matrix	coverage-matrix.json	5 games x 6 dimensions mapping
Metric Mapping	metric-mapping.json	15 metrics linked to telemetry events
ECD: Pulse Realms	pulse-realms.pdf	Full ECD model (Competency/Evidence/Task/Assembly)

ECD: Other Games	`docs/assessment/ecd/`	Draft ECD skeletons for 4 future games